RAM MEMORY ARCHITECTURE DESIGN

Contents

Introduction 2

RAM Category-Based Memory Systems 3

RAM Memory Architecture Memory Protection Systems 4

OS Management of RAM and Fallback Systems 5

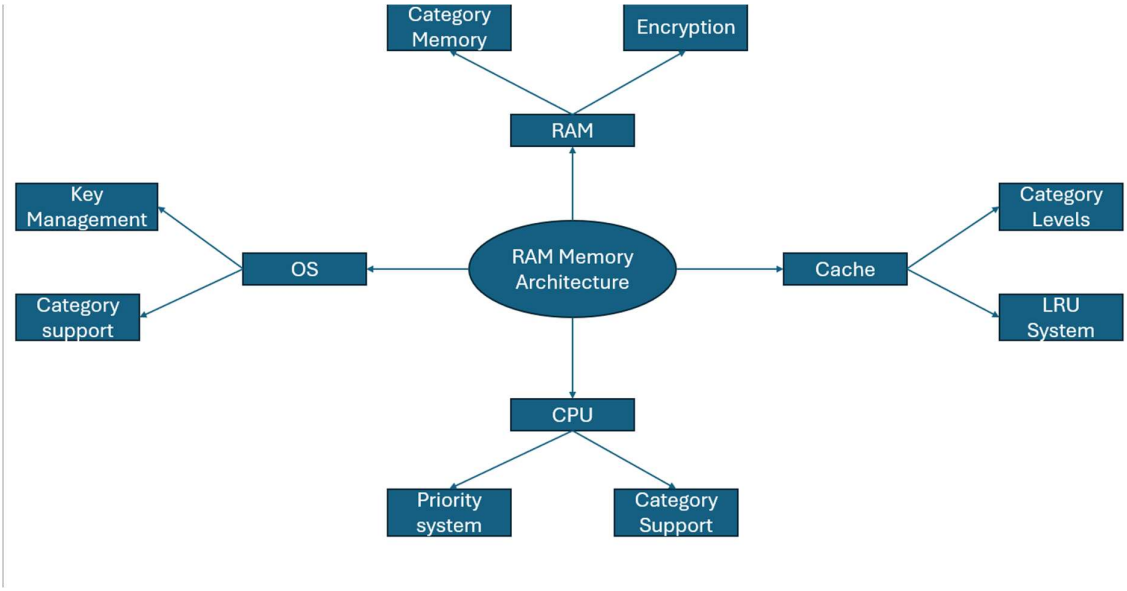
RAM Memory Architecture Workflow 6

RAM Memory Architecture Trade-offs 7

Introduction

This RAM Memory Architecture is designed to improve the performance and security of RAM. This was also designed to address the limitation of flat addressing, which RAM currently uses to store instructions. This RAM memory architecture was created because modern systems and functions such as gaming, AI/ML infrastructure, and server workloads require fast and consistent performance. This RAM Memory Architecture was created with two main features. These features are category-based memory and RAM encryption mode. This design will explain the changes needed and how each feature works.

This Memory Architecture cannot promise perfect performance or security of RAM. However, it can promise to increase performance and reduce workloads compared to the current flat addressing systems. It can also provide protection against cold boot attacks without having to write contents to encrypted disk. This is a design which may be time-consuming for engineers to implement. Some systems may only use the category-based memory or RAM encryption mode while some systems may use both. Make sure to adapt this design based on operating system and CPU architecture if necessary for your own systems.



This spider diagram shows the key changes made to RAM, the CPU, Cache and operating system. For those that may not be familiar with computing architecture, LRU stands for least recently used which is used to remove less frequently used data from cache. More detail on changes for cache can be found in the cache design document in this repo.

RAM Category-Based Memory Systems

- **Category-based memory is a system where data within RAM or any storage device is organised into clear labels which in this case are categories.**
- **There are 3 subsystems that are involved in category-based memory.**
- **The first subsystem is the category allocation within RAM.**
- **There are 3 categories used in total for RAM.**
- **The first category is the OS category which stores all operating system processes, instructions and data.**
- **The second category is the application/app category which stores all user application processes, instructions and data.**
- **The third category is the background category which stores all background processes, instructions and data.**
- **These 3 categories were chosen as they cover all the instructions, data and processes that would be realistically stored in RAM.**
- **The second subsystem is category-allocation system**
- **This system is designed to allocate the categories space within RAM within the physical slots.**
- **For the physical RAM slots, the OS will assign addresses and the amount of space per how much each RAM slot allows.**
- **However, not every system will have the physical RAM slots immediately available, so this system is well-designed for that.**
- **For addressing, the OS will assign each category a range of addresses meaning that every process, instruction and data will have to use an address within that.**
- **For logical space allocation, The OS will calculate how much space the OS category needs and allocates a fixed number of space for it with a 5% safety buffer.**
- **For the background category, the OS will allocate 5% of RAM storage for it as it isn't expected to change much.**
- **For the application category, the OS will allocate the remaining of RAM space for it as it requires the most storage within RAM.**
- **It is important to remember that the allocation logical system above is only for if the physical RAM slots aren't available.**
- **The third and final subsystem is data organisation within categories.**
- **This subsystem uses an algorithm I call the in-category-organisation algorithm which organises instructions, processes and data within each category.**
- **For caching, the algorithm will allocate 5% of category space to it to ensure that the CPU can still have quick access.**
- **For processes and instructions, the algorithm will allocate 15% to processes and 5% for instructions.**
- **For data, the rest of the space available within the category will be allocated which is about 75% of the space for this data.**

RAM Memory Architecture Memory Protection Systems

- **There are 4 subsystems used for the security of memory within RAM.**
- **The first subsystem is buffer overflow protection system.**
- **This system is designed to prevent attackers from making processes or memory regions from using more memory than expected.**
- **First, the operating system sets a fixed amount of memory for each process and memory region within a category.**
- **Second, the operating system consistently monitors the processes memory usage in real-time to ensure that it isn't using too much memory.**
- **Third, if a process or memory region is using too much memory, the operating system can cut off the process or limit how much memory the process can use.**
- **This system was chosen as it ensures that the system can function normally but also ensure that processes don't go over their assigned memory.**
- **The second subsystem is available memory protection system.**
- **First, the operating system keeps track of what memory regions or pointers are free within each category.**
- **Second, an authorised and verified service allocates these available memory regions and pointers to where they are needed.**
- **Third, if the operating system detects another service which is unauthorised then the service is shut down and the regions that were allocated are reallocated.**
- **Fourth, each process is limited to what it can access within its own category and within other categories to prevent movement of malicious processes.**
- **This system was chosen as it ensures that allocation can proceed as normal while also allowing the operating system to fight back against attackers.**
- **The third subsystem is pointers protection system.**
- **First, the operating system keeps a track of what each pointer is doing and what pointers are available.**
- **Second, the operating system creates signatures for each pointer to verify the authenticity of the pointer.**
- **Third, if a pointer is unverified within the memory than the operating system can investigate and if needed remove the pointer from memory.**
- **This system was chosen as it allows the operating system to use pointers normally while not allowing attackers to use their own malicious pointers.**
- **The fourth and final subsystem is RAM encryption mode.**
- **RAM encryption mode is a user-initiated state that encrypts all contents of RAM to protect from cold boot attackers while the device is unattended.**
- **The user can choose an available encryption algorithm such as AES-256 or Serpent which is attached to an authentication method like a PIN.**
- **The key is also stored within TPM which is a strict requirement for this mode.**
- **The user can use a recovery key or as a final fallback turn off the device to clear the contents of RAM.**

OS Management of RAM and Fallback Systems

- **The operating system needs to consistently and constantly manage RAM resources to perform its tasks effectively.**
- **There are 4 subsystems used in this management and fallback system.**
- **The first subsystem is paging system.**
- **First, the operating system creates virtual pages that can be moved between RAM and the disk of the device.**
- **Second, these virtual pages are marked with categories so the operating system or any logical component can know which category the page belongs to.**
- **Third, these virtual pages arrive in the correct area within the internal drive/disk of the device.**
- **This system was chosen as it makes the paging process that the operating system uses more category aware.**
- **The second subsystem is for swapping.**
- **First, the operating system detects that a category within RAM is full and can't store anymore instructions, data or processes.**
- **Second, the operating system mounts the corresponding partition within the internal drive and begins the swapping process.**
- **Third, the virtual pages are swapped between a category in RAM and the same category in the internal drive just as a partition.**
- **Fourth, after space is available, the operating system returns to normal function meaning swapping has stopped.**
- **This system was chosen as it makes the swapping process category aware.**
- **The third subsystem is caching.**
- **First, the operating system allocates a small region of memory within each category for storing the most frequently used files.**
- **Second, the operating system uses LRU for this memory region to allow for only the most frequently used data to remain in that cache.**
- **Third, for the operating system category the write-through system is used as it is safer especially for such critical data as the operating system.**
- **Fourth, for the application and background categories, the write-back system is used for speed over safer as the data isn't as critical.**
- **This system was chosen as caching is important for speed as well as for ensuring that data is stored within multiple locations.**
- **The fourth and final subsystem is sorting validation system.**
- **First, the operating system memory manager sorts out the data in RAM into categories using address ranges and other methods.**
- **Second, a secondary service verifies the sorting of the data and alerts the memory manager of any mistakes it could have made. This has read-only access.**
- **Third, this secondary service also runs in real-time and verifies that the category-aware algorithm organised the data correctly in each category.**

RAM Memory Architecture Workflow

- **This section has three workflows which shows the architecture during normal operation, swapping and during the RAM encryption mode.**
- **First, the program counter points to the next category and address within memory.**
- **This address within the program counter could be OS: 0x7C1E which is specific to the OS category's program counter.**
- **Second, the category and the address are temporarily stored within the memory address register.**
- **Third, the address bus navigates to the OS category within RAM and finds the address within that category.**
- **Fourth, the data bus transports the instruction to the MDR which is also known as memory data register.**
- **Fifth, the instruction is then sent to the current instruction register also known as the CIR.**
- **In the CIR, the instruction is decoded/split into operand and opcode to be executed.**
- **The opcode specifically tells the CPU what to do with the data, while the operand tells the CPU what data to perform that action on.**
- **Sixth, the CPU executes the instructions using other components such as the GPU when needed.**
- **Finally, the program counter increments by one and points to the next category and address in memory**
- **For example, the app program counter may display "App: 0x4F2A"**
- **For the second workflow, we start off with the user running multiple heavy applications, tabs and processes.**
- **Second, the operating system immediately begins the swapping process to keep the device running.**
- **Third, the operating system category in RAM swaps pages with the OS partition within the disk.**
- **Fourth, the application category in RAM swaps pages with the app partition within the disk.**
- **Fifth, these swapping operations continue until the heavy workloads have stopped.**
- **Sixth, the RAM storage within each category is returned to normal and no swapping is done.**
- **For the third workflow, we start off with the user selecting AES-256 as the encryption algorithm and using a custom PIN for the mode.**
- **User sets up recovery key and stores it in a different location**
- **Second, the device enters a power saving mode and encrypts the RAM contents with the key stored within TPM.**
- **Finally, the user comes back, authenticates and begins normal operation again.**

RAM Memory Architecture Trade-offs

- **This RAM memory architecture has trade-offs just like any other system I have built.**
- **There are 5 main trade-offs that I have identified.**
- **Engineers may face more trade-offs during implementation as this is only a design.**
- **The first trade-off is compatibility.**
- **This is because the encryption mode may not be compatible with mobile device due to the ARM architecture.**
- **The encryption mode also requires the device to have an TPM which not all devices support.**
- **This trade-off can be managed by targeting PC/laptops, using other secure storage methods on the OS or on the cloud and focusing on encryption mode for the targeted audience.**
- **The second trade-off is performance.**
- **This is because the encryption mode can cause latency due to encrypting and decrypting the contents of RAM especially if the RAM size is large.**
- **This trade-off can be managed by keeping the encryption mode strictly to hibernation mode and using pre-existing operating system authentication.**
- **The third trade-off is lockout risk.**
- **This is because the encryption mode may lockout the user if they forget their password.**
- **This trade-off can be managed through the recovery key (No data lost) or turning off the device which means all data in RAM is lost.**
- **The fourth trade-off is heat.**
- **This is because since the CPU can fetch instructions more quickly it can also execute more instructions per cycle.**
- **This can cause the CPU to heat up a lot more than flat addressing.**
- **This trade-off can be managed by cooling systems and clock speed within the CPU.**
- **The fifth trade-off is legacy support.**
- **This is because this RAM memory architecture requires major changes for the CPU, updates to the operating system and changes to RAM.**
- **This can take years, and most users will not be able to adapt from current hardware easily.**
- **This trade-off can be managed through new operating system memory controller , new generation of CPU and a new RAM module.**
- **This legacy support trade-off will be the most difficult and expensive to migrate.**
- **There may be some more migrations that can be used for these trade-offs, I have not considered.**
- **Make sure to adapt migrations based on resources and budget.**